

ARM PrimeCell

SDRAM Controller (PL170)

User Guide

ARM PrimeCell SDRAM Controller (PL170) User Guide

© Copyright ARM Limited 2000. All rights reserved.

Proprietary notice

ARM, the ARM powered logo, Thumb and StrongARM are registered trademarks of ARM Limited.

The ARM logo, PrimeCell, AMBA, Angel, ARMulator, EmbeddedICE, ModelGen, Multi-ICE, ARM7TDMI, ARM9TDMI, TDMI and STRONG are trademarks of ARM Limited.

All other products or services mentioned herein may be trademarks of their respective owners.

Neither the whole nor any part of the information contained in, or the product described in, this document may be adapted or reproduced in any material form except with the prior written permission of the copyright holder.

The product described in this document is subject to continuous developments and improvements. All particulars of the product and its use contained in this document are given by ARM Limited in good faith. However, all warranties implied or expressed, including but not limited to implied warranties or merchantability, or fitness for purpose, are excluded.

This document is intended only to assist the reader in the use of the product. ARM Limited shall not be liable for any loss or damage arising from the use of any information in this document, or any error or omission in such information, or any incorrect use of the product.

Document confidentiality status

This document is Open Access. This document has no restriction on distribution.

Product status

The information in this document is Final (information on a developed product).

Web address

<http://www.arm.com>

Feedback

ARM limited welcomes feedback on both the product, and the documentation.

Feedback on this document

If you have any comments on about this document, please send email to errata@arm.com giving:

- The document title
- The documents number
- The page number(s) to which your comments refer
- A concise explanation of your comments

General suggestion for additions and improvements are also welcome.

Feedback on this product

If you have any comments or suggestions about this product, please contact your supplier giving:

- The product name
- A concise explanation of your comments.

Contents

1	ABOUT THIS DOCUMENT	1
1.1	Change History	1
1.2	References	1
1.3	Terms and abbreviations	1
2	SCOPE	2
3	DIRECTORY STRUCTURE	3
3.1	PrimeCell SDRAMC peripheral directory structure	3
3.2	Synopsys synthesis reference directory structure	4
3.3	BusTalk testbench directory structure	5
3.4	MODELS directory structure	6
4	USAGE INSTRUCTIONS	7
4.1	Introduction	7
4.2	Design tools used	7
4.3	Reference Cell library	7
4.4	Simulation with ModelSim and Synthesis using Design Compiler	9
4.4.1	Modification of environment variables	9
4.4.2	Compiling the Peripheral VHDL RTL Code	10
4.4.3	Compiling the Peripheral Verilog RTL Code	10
4.4.4	Generating BusTalk test vectors	11
4.4.5	Compiling the BusTalk Testbench	12
4.4.6	BusTalk testbench simulation of Peripheral RTL	13
4.4.7	Synthesis using Synopsys Design Compiler	14
4.4.8	BusTalk testbench gate-level simulation	16
4.4.9	ModelSim error and warning messages	17
	Simulation with Cadence Verilog-XL	18
4.4.10	Modification of environment variables	18
4.4.11	Generating BusTalk test vectors	18
4.4.12	Linking Cadence to Denali models	18
4.4.13	BusTalk testbench simulation of Peripheral RTL	18
4.4.14	BusTalk testbench gate-level simulation	19
4.4.15	Verilog-XL error and warning messages	21

1 ABOUT THIS DOCUMENT

1.1 Change History

Issue	Date	Change
A	6 December, 1999	First release.
B	26 January, 2000	Add more testbench information (section 4.4.4) minor corrections.
C	10 April, 2000	Add sections on Denali memory models

1.2 References

For further information refer to the following documents.

Ref	Doc No	Title
1	ARM IHI 0011	AMBA Specification (Rev 2.0)
2	ARM DDI 0006	AMBA Compliance Test Suite (bus compliance tests)

1.3 Terms and abbreviations

This document uses the following terms and abbreviations.

Term	Meaning
AHB	Advanced High-performance Bus
AMBA	Advanced Micro-controller Bus Architecture
APB	Advanced Peripheral Bus
ASB	Advanced System Bus
ASIC	Application Specific Integrated Circuit
IP	Intellectual Property
RTL	Register Transfer Level
SDRAM	Synchronous Dynamic Random Access Memory

2 SCOPE

This document gives instructions for the installation and use of the ARM PrimeCell SDRAM Controller (SDRAMC) PL170.

For further information on AMBA, refer to the AMBA Specification [Ref.1].

The AMBA Bus Compliance Test Suite [Ref. 2] contains information on test methodology using AMBA bus compliance testbenches.

3 DIRECTORY STRUCTURE

The PrimeCell SDRAM Controller peripheral is organized using a standard directory structure used with other AMBA peripherals which use an AMBA-based verification test methodology.

3.1 PrimeCell SDRAMC peripheral directory structure

Figure 3-1 illustrates the peripheral directory structure for the ARM PrimeCell SDRAMC peripheral:

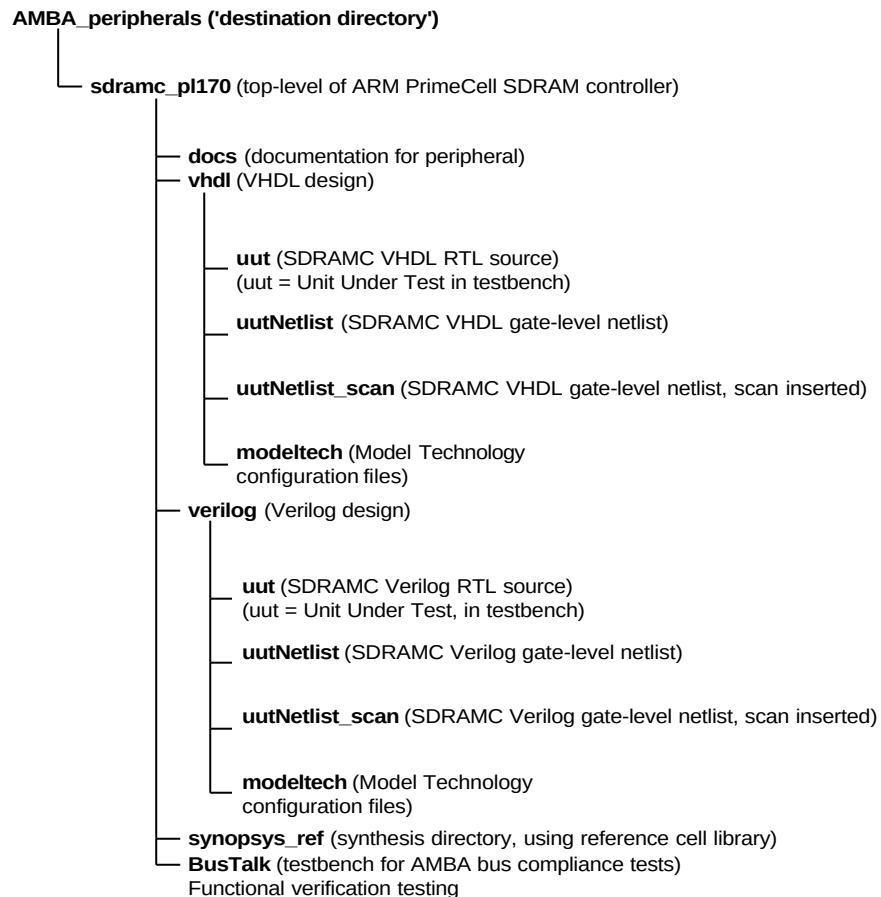


Figure 3-1. PrimeCell SDRAMC peripheral directory structure

The VHDL and Verilog RTL code is located in the `sdramc_PL170/vhdl/uut` and `sdramc_PL170/verilog/uut` directories respectively. Documentation is located in the `sdramc_PL170/docs` directory.

3.2 Synopsys synthesis reference directory structure

The `synopsys_ref` directory contains files and directories for synthesis of the VHDL and Verilog RTL targeted to a reference cell library that is supplied with the PrimeCell SDRAMC peripheral. This allows the user to validate synthesis and gate-level simulation of the PrimeCell SDRAMC design, or compare results using different configuration values and alternative design tools.

Figure 3-2 illustrates the Synopsys synthesis reference directory structure:

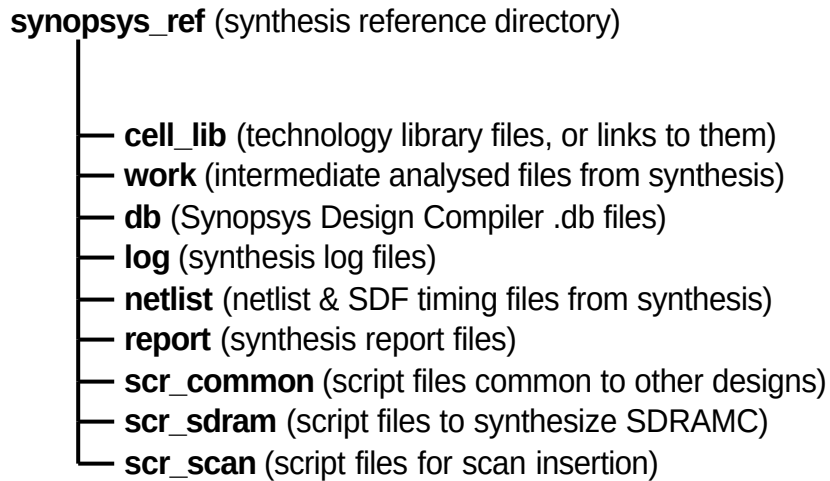


Figure 3-2. Synopsys synthesis reference directory structure

3.3 BusTalk testbench directory structure

The BusTalk directory contains the standalone verification testbenches in both VHDL and/or Verilog. A 'C' test program in the bustests directory is used to produce BusTalk testbench input files which are created in bustests/invec. The input files are used for functional verification of the PrimeCell SDRAMC macro blocks by applying AMBA bus transfers.

Figure 3-3 illustrates the BusTalk testbench directory structure:

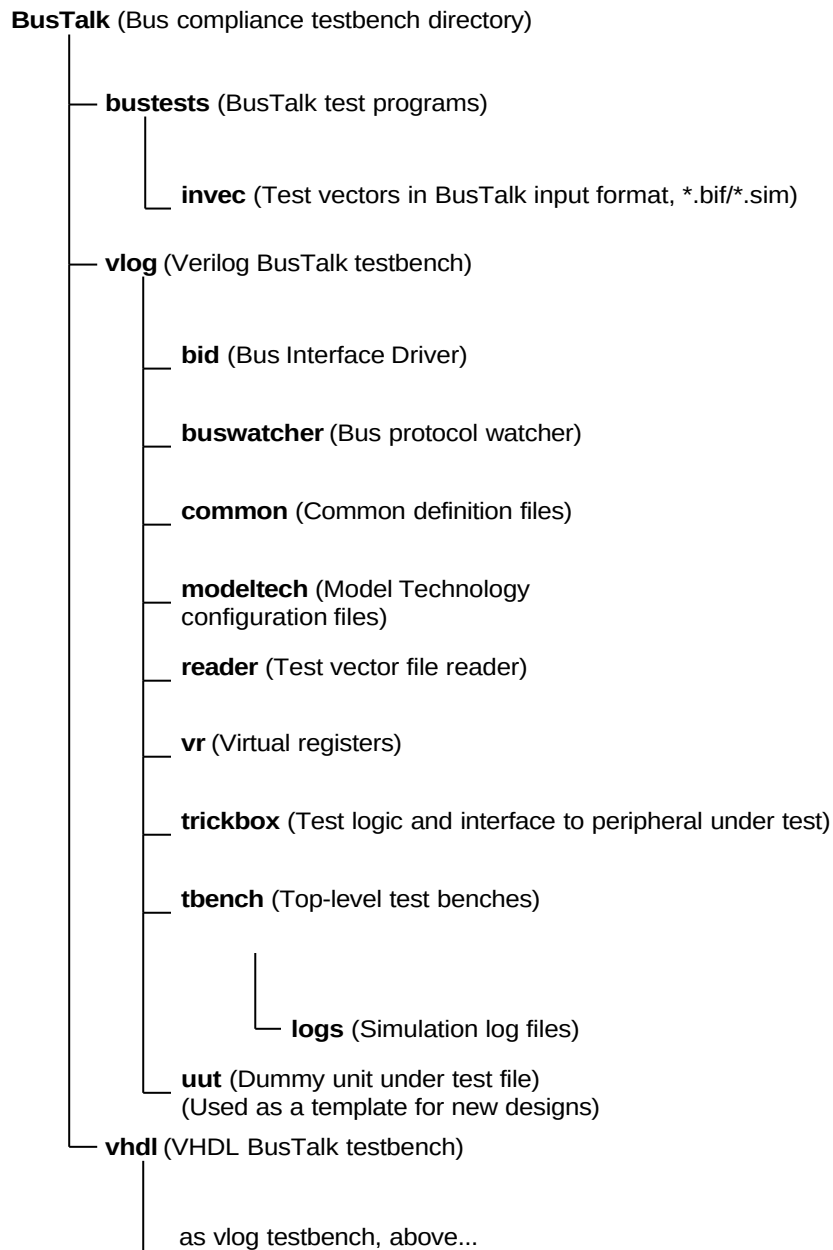


Figure 3-3. BusTalk testbench directory structure

3.4 MODELS directory structure

A separate directory tree is supplied containing a collection of memory models, these require the Denali memory modeler tool. The Denali memory modeler tool is not supplied. To allow verification simulations using the Denali models supplied by ARM, Denali have agreed to supply the memory modeler tool and a temporary licence.

Contact Denali to obtain the memory modeler tool, their web site is at <http://www.denalisoft.com>

MODELS (Top level models directory)

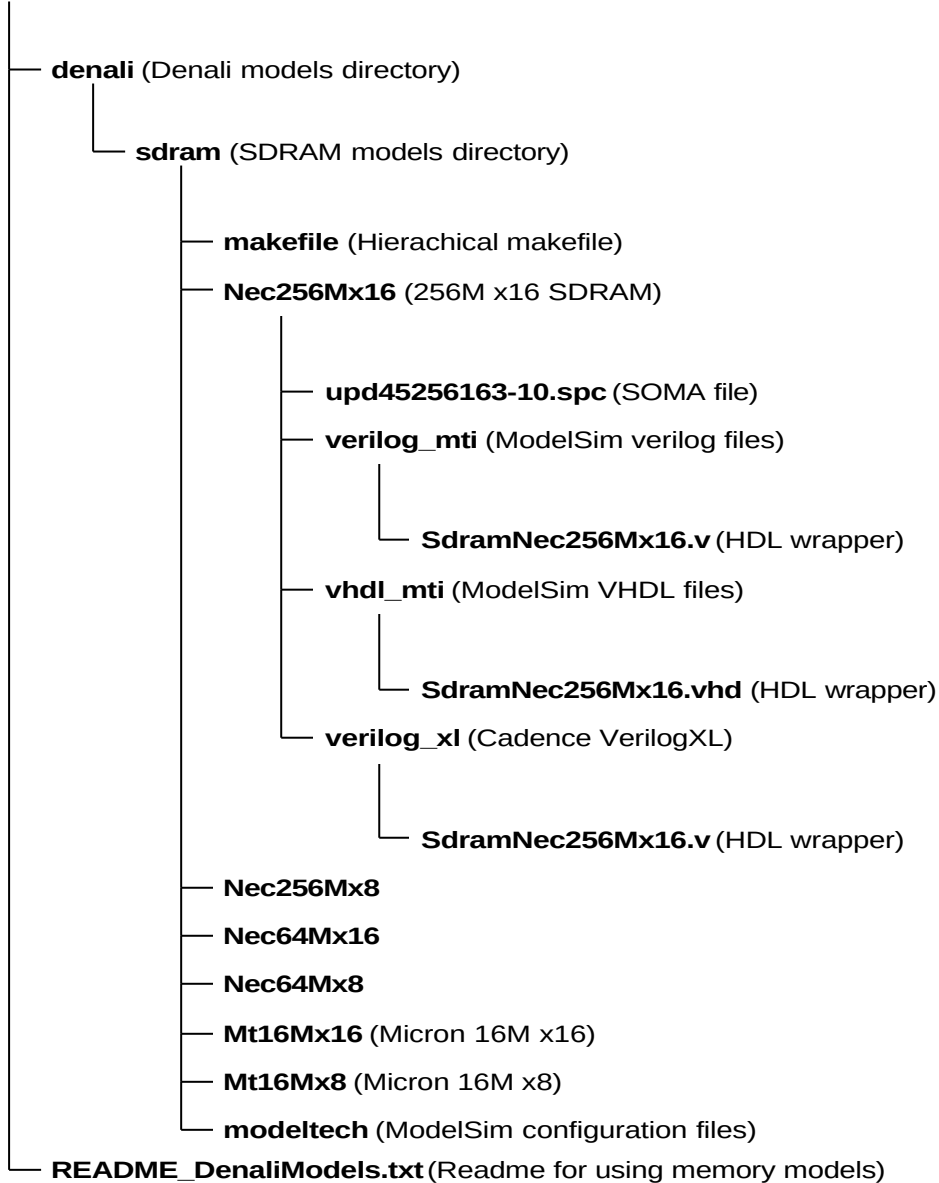


Figure 3-4. MODELS directory structure

4 USAGE INSTRUCTIONS

4.1 Introduction

This PrimeCell SDRAMC peripheral has been designed in both synthesizable VHDL and Verilog Hardware Description Languages (HDL).

Standalone functional verification test benches are supplied using AMBA Bus Compliance (BusTalk) methodology.

4.2 Design tools used

The release note for the PrimeCell SDRAMC peripheral describes the simulation and synthesis tools used. When different design tools are used, amendments to the configuration, script and make files will be required.

The simulation examples below illustrate the use of Model Technology Inc, ModelSim commands for both VHDL and Verilog designs and Cadence Verilog-XL for the Verilog designs. Amend the examples as appropriate if different simulation tools are used.

VHDL and Verilog simulations have been performed with Model Technology Inc, ModelSim. The ModelSim simulator uses a 'libmap' file in the BusTalk/vlog/modeltech and BusTalk/vhdl/modeltech directories to map the chosen library to the RTL (Verilog or VHDL) or the gate-level netlist. This reduces editing of the HDL files. ModelSim is an event-driven simulator which produces compiled environments for both VHDL and Verilog.

Verilog simulations have also been performed using Cadence Verilog-XL. Verilog-XL is an event-driven simulator which interprets the HDL at run-time.

For further information, refer to the README files within the tbench directories for each testbench.

Example simulation log files (transcripts) are located in the BusTalk/<vlog|vhdl>tbench/logs directories.

4.3 Reference Cell library

The Avant! CB25 (0.25 μ m) standard cell library is supplied with the PrimeCell SDRAMC peripheral as a reference cell library. This proprietary Avant! Passport standard cell library has been used during development of the peripheral, for synthesis and gate-level simulation with post-synthesis back-annotated timings in Standard Delay Format (SDF). For further information on this cell library, refer to the documentation supplied.

Use of a different technology library or design tools requires that script files and configuration files must be amended for the user's chosen technology library, simulation and synthesis tools.

For further information, particularly regarding amendment of synthesis script files to a different technology library, and integration of peripherals in an AMBA-based ASIC, refer to the Integration Manual.

The availability of this reference cell library provides a means for the end user to replicate the synthesis and verification processes performed prior to delivery. Please note that this is a reference library only with no physical views, and users must provide their own target library for the development of their system.

The gate count and performance information relate to the reference library only. It is believed that this library is representative of a typical vendor cell library. Synthesis to the customers target library should yield similar gate counts, but performance is vendor library specific.

Figure 4.3-1 illustrates directory structure that has been used for development of the ARM PrimeCell SDRAMC and environment variables used by UNIX make files default to this structure.

The top-level Avant! CB25 directory is expected to be at the same level as the top-level of the PrimeCell SDRAMC peripheral, in order to run the gate-level simulations as defined in configuration files, shell scripts and usage instructions described later. If a different directory structure is used the files must be amended accordingly.

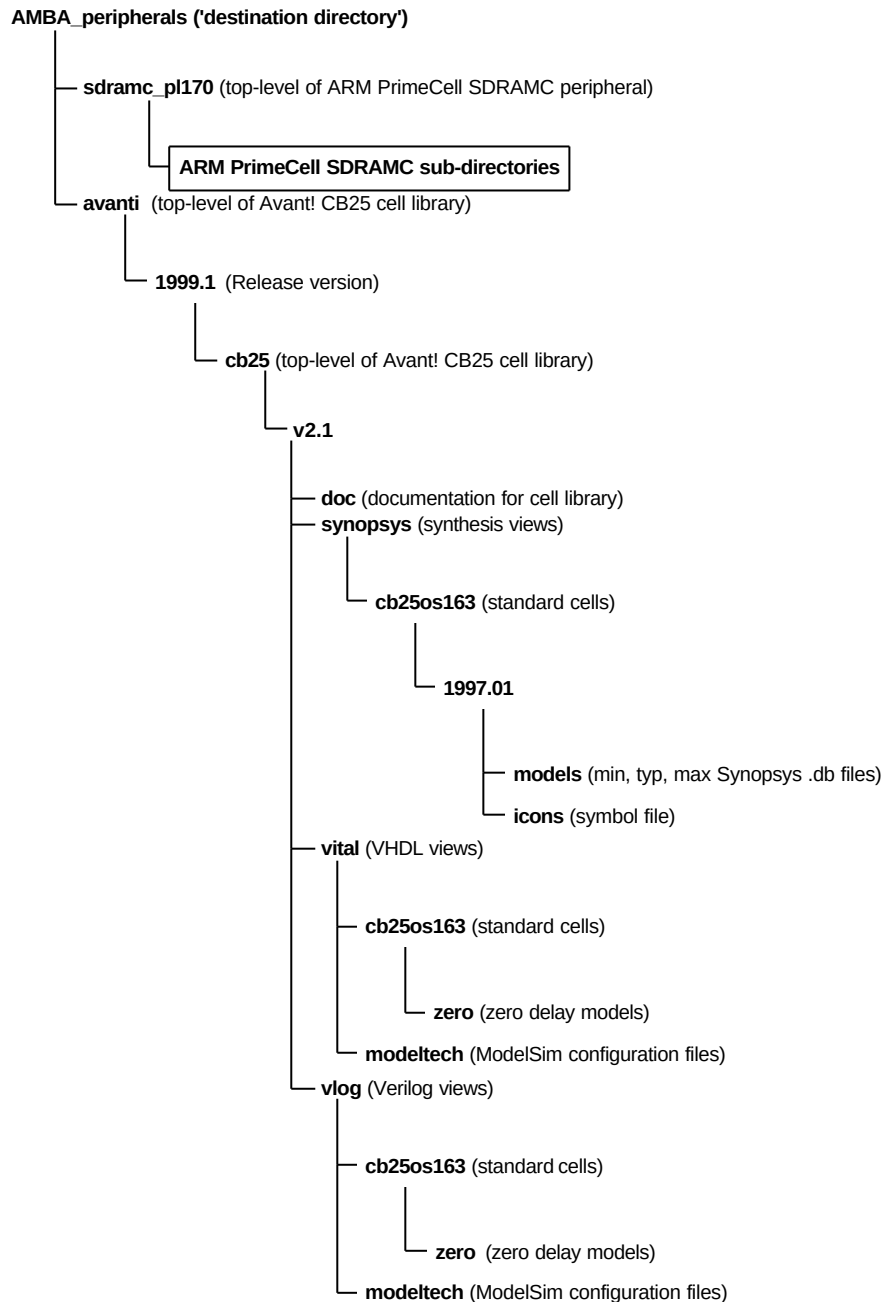


Figure 4.3-1 Avant! CB25 cell library directory structure

If the ModelSim simulator is to be used, create directories for modeltech configuration files following the structure shown above. Makefiles or compilation shell scripts also need to be amended for the chosen cell library.

4.4 Simulation with ModelSim and Synthesis using Design Compiler

4.4.1 Modification of environment variables

UNIX environment variables have been used to simplify configuration of directory paths in make files, synthesis scripts and simulation configuration files.

The environment variable "PERIPH" is used to access the top directory level of the peripheral. To set this environment variable, change to the top-level directory of the peripheral and set the PERIPH environment variable to the current working directory (\$cwd for C shell):

```
cd <AMBA_peripherals>/sdramc_pl170
setenv PERIPH $cwd
```

where <AMBA_peripherals> is the absolute directory path to the destination directory and sdramc_pl170 is the peripheral top-level directory name.

Simulations make use of Denali memory models referenced using the environment variable MODELS. The environment variable MODELS should be set to the path of the MODELS directory. Memory references use a path like \$MODELS/Denali/sdram/<memorytype>. Where <memorytype> is the directory containing files specific to a memory device. Each memory device directory contains a Denali soma file and directories containing material specific to each simulator language and the related work directories.

The above commands could be written in a script file (for example a file named SourceMe in the top-level peripheral directory) and executed using the UNIX source command as below:

```
source SourceMe
```

Note the SourceMe file must be sourced at the beginning of each xterm session.

The SourceMe file could also source setup files for the design tools being used

An example SourceMe file is shown below, but the directory paths need to be amended:

```
# Set up variables specific to this peripheral macro block
# -----
# Set the path environment variable to the peripheral design root directory
# For example sdramc_pl170/
setenv PERIPH /home/joe/work/sdramc_pl170
setenv MODELS /home/joe/work/MODELS

# Set the peripheral name abbreviation, for example Sdramc
setenv PERIPHERAL Sdramc

# Set the ARM Product number for this peripheral, for example pl170
setenv PRODUCT pl170

# VHDL Cell library
# -----
# Set the path environment variable to the technology library VITAL timing views
# For VHDL gate level simulations use LIB_VITAL

# Set the generic library path for the Avant! CB25 VITAL library
setenv CB25VITAL $PERIPH/../../LIBRARIES/avanti/1999.1/cb25/v2.1/vital

# Set the path to the Avant! CB25 VITAL library, standard cell zero delay view
setenv LIB_VITAL $CB25_VITAL/cb25os163/zero

# VERILOG Cell library
# -----
```

```
# Set the path environment variable to the technology library Verilog timing views
# For Verilog gate level simulations use LIB_VERILOG

# Set the generic library path for the Avant! CB25 Verilog library
setenv CB25VERILOG $PERIPH/../../LIBRARIES/avanti/1999.1/cb25/v2.1/verilog

# Set the path to the Avant! CB25 VITAL library, standard cell zero delay view
setenv LIB_VERILOG $CB25_VERILOG/cb25os163/zero
```

4.4.2 Compiling the Peripheral VHDL RTL Code

Ensure the environment variable PERIPH is set or modify the directory paths.

The SDRAMC VHDL is located in the \$PERIPH/vlog/uut directory, where uut refers to the Unit Under Test within the testbench structure.

To compile the VHDL code using make files supplied, change to the vhd/uut/ directory:

```
cd $PERIPH/vlog/uut/
```

and execute the make file (called makefile) to compile the Peripheral VHDL RTL sub-blocks, in the correct compilation order:

```
make clean
```

4.4.3 Compiling the Peripheral Verilog RTL Code

Ensure the environment variable PERIPH is set or modify the directory paths.

The SDRAMC Verilog is located in the \$PERIPH/verilog/uut directory, where uut refers to the Unit Under Test within the testbench structure.

To compile the Verilog code using make files supplied, change directory to the verilog/uut/ directory:

```
cd $PERIPH/verilog/uut/
```

and execute the make file (called makefile) to compile the Peripheral Verilog RTL sub-blocks, in the correct compilation order:

```
make clean
```

4.4.4 Generating BusTalk test vectors

Ensure the environment variable PERIPH is set or modify the directory paths. Ensure suitable memory models are present, compiled and the MODELS variable is set, or modify the directory paths in the test files.

BusTalk test compliance programs are written in 'C' and located in the \$PERIPH/BusTalk/bustests directory.

There are five test sets, each uses four 'C' test files, each file in a set generates the test sequence for one of the four AHB ports. The following table shows the testbench used when running a 'C' test compliance program and the corresponding BusTalk test vector stimulus files (.bif or .sim) generated by the make process.

'C' test compliance programs Path: BusTalk/bustests/	Test vectors Path: BusTalk/bustests/invec/ Verilog testbench vectors are type .sim VHDL testbench vectors are type .bif		Testbenches Path, VHDL: BusTalk/vhdl/tbench/ Path, Verilog: BusTalk/vlog/tbench/
Sdram3.c Sdram2.c Sdram1.c Sdram0.c	Sdram3.bif Sdram2.bif Sdram1.bif Sdram0.bif	Sdram3.sim Sdram2.sim Sdram1.sim Sdram0.sim	tb_Sdram.vhd (VHDL) tb_Sdram.v (Verilog)
Sdram_256M_3.c Sdram_256M_2.c Sdram_256M_1.c Sdram_256M_0.c	Sdram_256M_3.bif Sdram_256M_2.bif Sdram_256M_1.bif Sdram_256M_0.bif	Sdram_256M_3.sim Sdram_256M_2.sim Sdram_256M_1.sim Sdram_256M_0.sim	tb_Sdram_256M.vhd (VHDL) tb_Sdram_256M.v (Verilog)
Sdram_CAS2_3.c Sdram_CAS2_2.c Sdram_CAS2_1.c Sdram_CAS2_0.c	Sdram_CAS2_3.bif Sdram_CAS2_2.bif Sdram_CAS2_1.bif Sdram_CAS2_0.bif	Sdram_CAS2_3.sim Sdram_CAS2_2.sim Sdram_CAS2_1.sim Sdram_CAS2_0.sim	tb_Sdram.vhd (VHDL) tb_Sdram.v (Verilog)
Sdram_CAS2_256M_3.c Sdram_CAS2_256M_2.c Sdram_CAS2_256M_1.c Sdram_CAS2_256M_0.c	Sdram_CAS2_256M_3.bif Sdram_CAS2_256M_3.sim Sdram_CAS2_256M_2.bif Sdram_CAS2_256M_2.sim Sdram_CAS2_256M_1.bif Sdram_CAS2_256M_1.sim Sdram_CAS2_256M_0.bif Sdram_CAS2_256M_0.sim		tb_Sdram_256M.vhd (VHDL) tb_Sdram_256M.v (Verilog)
Sdram_CAS1_3.c Sdram_CAS1_2.c Sdram_CAS1_1.c Sdram_CAS1_0.c	Sdram_CAS1_3.bif Sdram_CAS1_2.bif Sdram_CAS1_1.bif Sdram_CAS1_0.bif	Sdram_CAS1_3.sim Sdram_CAS1_2.sim Sdram_CAS1_1.sim Sdram_CAS1_0.sim	tb_Sdram_CAS1.vhd (VHDL) tb_Sdram_CAS1.v (Verilog)

Table 4.4-1: BusTalk tests

Due to the large file sizes the BusTalk .bif and .sim files are not included in the peripheral release. To generate new versions of the BusTalk test vector .bif and .sim files, follow the instructions below:

Use the make file supplied in the BusTalk/bustests directory of the PrimeCell SDRAMC peripheral for generation of the BusTalk input files.

To create BusTalk test vector files, ensure that the "#define TIF" is commented out `/* #define TIF */`, in the config.h file (modify and save if not), and execute the UNIX command below:

```
make <test_file>
```

Repeat for each test file e.g. 'make Sdram0', 'make Sdram1', 'make Sdram2' and 'make Sdram3'. Note .c is omitted from the file name specified in the command line.

The UNIX environment variable VECT_TYPE should be undefined before creating the BusTalk vectors.

The .bif and .sim files are generated in the \$PERIPH/BusTalk/bustests/invec/ directory.

4.4.5 Compiling the BusTalk Testbench

Ensure the environment variables PERIPH and MODELS are set or modify the directory paths.

Ensure that \$PERIPH/BusTalk/vlog/modeltech/libmap maps uut to the compiled VHDL or Verilog RTL

```
uut = $PERIPH/vlog/uut/work
or
uut = $PERIPH/verilog/uut/work
```

The BusTalk VHDL testbench is located in the \$PERIPH/BusTalk/vhdl/ directory.

To compile the BusTalk VHDL testbench code with make files supplied, change to the \$PERIPH/BusTalk/vhdl/ directory:

```
cd $PERIPH/BusTalk/vhdl/
```

and execute the make file (called makefile) to compile the BusTalk testbench in the correct compilation order.

```
make clean
```

To compile the BusTalk Verilog testbench code using the make files supplied, change to the \$PERIPH/BusTalk/vlog/ directory:

```
cd $PERIPH/BusTalk/vlog/
```

and execute the make file (called makefile) to compile the BusTalk testbench in the correct compilation order.

```
make clean
```

- Note** For successful compilation of the top-level testbenches, the appropriate VHDL or Verilog RTL, or VHDL or Verilog netlist must have been previously compiled and pointed to within the modeltech/libmap or simulator-specific configuration file.
- Note** To perform gate-level simulations, synthesis must be performed to create a netlist in the required format. Some post-processing of the netlist and/or timing files may be needed for compatibility with the technology library. Compile the cell library HDL files first, then the netlist and finally the testbench.
- Note** Compilation of the Verilog testbench must be done after any change to the test vector file link (*.sim), as the file pointer is only updated during compilation for Verilog, whilst it occurs at run-time for VHDL.
- Note** To enable generation of a toggle report in ModelSim using a startup command file, it is necessary to replace \$finish by \$stop in the Verilog reader.v file in the reader directory.

4.4.6 BusTalk testbench simulation of Peripheral RTL

Ensure the environment variables PERIPH and MODELS are set or modify the directory paths.

Ensure links are set appropriately (to map the peripheral HDL RTL) and the BusTalk testbench and memory models are compiled.

Ensure that the appropriate test vector file exists in \$PERIPH/BusTalk/bustests/invec.

For VHDL:

Ensure four links exist in the tbench directory to the test vector files \$PERIPH/BusTalk/bustests/invec/infile0.bif, infile1.bif, infile2.bif and infile3.bif, the example below is for the Sdram0.bif test vector file, substitute the names corresponding to the set of test vector files to be run, to create the four links:

```
cd $PERIPH/BusTalk/vhdl/tbench
ln -s ../../bustests/invec/Sdram0.bif infile0.bif
```

For Verilog:

Ensure four links exist from the reader directory to the .sim test vector files in \$PERIPH/BusTalk/bustests/invec, the example below is for the Sdram0.sim test vector file, substitute the names corresponding to the set of test vector files to be run, to create the four links:

```
cd $PERIPH/BusTalk/vlog/reader
ln -s ../../bustests/invec/Sdram0.sim infile0.sim
```

Verilog requires the HDL to be compiled after the links to the *.sim testbench input files have been created.

To perform a simulation with the functional verification testbench using ModelSim, invoke the ModelSim simulator from the tbench directory:

```
vsim tb_Sdram
```

Entering a run -all command in the simulation window will cause the whole of the test pattern to run. The simulation will terminate correctly with the following message to signal completion of the test:

**** Failure: End of test (VHDL) or End of test (Verilog)**

There should be no errors reported during the simulation for a successful run, except during the initialization period. It is advised that simulation transcript files are always checked for any errors or warnings during any subsequent simulations.

Example simulation log files are supplied and are located in the \$PERIPH/BusTalk/<vhdl|vlog>/tbench/logs directory. The log files have been generated from simulations of the RTL and netlist for both VHDL and Verilog versions of the peripheral exercised with the functional verification test vectors.

4.4.7 Synthesis using Synopsys Design Compiler

The `synopsys_ref` directory contains a reference set of synthesis scripts for Synopsys Design Compiler which target the Avant! CB25 standard cell library. Refer to the above section regarding organization of the `synopsys_ref` directory structure.

To target your chosen cell library copy and rename the `synopsys_ref/` directory tree to `synopsys/` and then make amendments as required.

Ensure the environment variables `PERIPH` and `MODEL` are set or modify the directory paths.

Before synthesizing the peripheral, synthesis scripts will need to be modified to select the required cell library and define directory paths. For further information on modification of the synthesis scripts, refer to the Integration Manual supplied for this peripheral.

The setup script, `setup.scr` in the sub-directory `./scr_common`, defines the following key parameters

- the target synthesis cell library (using variables defined in `cb25v2.1settings.scr`)
- library cells to exclude during synthesis, for example, latches, JK and falling edge D flip-flops (defined by the include file `cb25dontuse.scr`)
- location of the Synopsys read and write cache

The above will need modifying to suit the choice of cell library and for the specific application. Refer to comments in the `setup.scr` script file for other parameters that might also need modifying to suit the design flow being implemented.

The global script, `global.scr` in `./scr_common` uses parameters from the cell library settings file, `cb25v2.1settings.scr`. These parameters need modifying to target a different cell library, to define:

- worst and best case conditions of the cell library
- default cell drive and load strength

The script file `cb25dontuse.scr`, in `scr_common`, specifies cells that should not be used for synthesis. The script `cb25scandontuse.scr`, in `scr_scan`, specifies cells that should not be used for scan replacement of sequential cells.

The directory `cell_lib` should contain links (or the files) for the target technology cell library, in Synopsys format.

For example, to use Avant! CB25, invoke the following UNIX commands in the `synopsys_ref/cell_lib` directory:

```
In -s ../.././avanti/1999.1/cb25/v2.1/synopsys/cb25os163/1997.01/models/cb25os163_max.db
In -s ../.././avanti/1999.1/cb25/v2.1/synopsys/cb25os163/1997.01/models/cb25os163_typ.db
In -s ../.././avanti/1999.1/cb25/v2.1/synopsys/cb25os163/1997.01/models/cb25os163_min.db
In -s ../.././avanti/1999.1/cb25/v2.1/synopsys/cb25os163/1997.01/icons/cb25os163.sdb
```

The timing parameters for the AMBA AHB interface are defined by `ahb_params.scr` and the constraints set in `ahb_slave.scr` in `./scr_common`. The timing constraints need to be modified for each design application.

The master compile script, `sdram.cmd` is located in the `./scr_sdram` directory. This script controls the synthesis process. No scan insertion is performed by this script, this is done in a separate script described later.

The timing constraints file for the non-AMBA inputs and outputs of the SDRAM, `sdram.scr` is located in the `./scr_sdram` directory and the constraints must also be modified for each design application.

The scripts require environment variables to be declared before starting synthesis, and these are described below.

The script variables `hdl` and `hdl_out` are assigned to UNIX environment variables `$HDL_IN` and `$HDL_OUT`, and these must be declared before beginning synthesis.

For example

```
setenv HDL_IN vhd1  
setenv HDL_OUT vhd1
```

or

```
setenv HDL_IN verilog  
setenv HDL_OUT verilog
```

The variable `hdl` selects the hdl source being compiled.

The variable `hdl_out` controls the output format of the netlist.

Alternatively, the script can be amended to define `hdl` and `hdl_out` without use of environment variables. If so desired, it is allowable to use VHDL source and output in Verilog format, or vice-versa.

The variable `design_root` specifies the root directory path of the design using the environment variable `$PERIPH`.

If environment variables are not required then enter a fixed path for `design_root`.

To synthesize the peripheral, change to the synopsys directory:

```
cd $PERIPH/synopsys_ref
```

To perform synthesis using Design Compiler in batch mode:

```
dc_shell -checkout "VHDL-Compiler" < scr_sdram/sdram.cmd > log/Sdram_vhd1.log  
or  
dc_shell -checkout "HDL-Compiler" < scr_sdram/sdram.cmd > log/Sdram_verilog.log
```

An additional compile script `sdram_scan.cmd` in `scr_scan/` allows scan insertion of the design as a post-synthesis process. The compile script `sdram.cmd` must be run first to obtain a `.db` file (located in the `db/` directory/) which has mapped and optimized the design to the chosen cell library. The `sdram_scan.cmd` compile script may then be used to replace sequential cells by their scan equivalents from the cell library.

To perform scan insertion using Design Compiler in batch mode, change to the synopsys directory, type:

```
dc_shell -checkout "VHDL-Compiler Test-Compiler" < scr_scan/sdram_scan.cmd >  
log/Sdram_scan_vhd1.log  
or  
dc_shell -checkout "HDL-Compiler Test-Compiler" < scr_scan/sdram_scan.cmd >  
log/Sdram_scan_verilog.log
```

When scan insertion occurs, sequential elements are replaced by scan equivalents, and the scan structure optimized to remove design rule violations. Should it be found that additional compilation is needed, this could be eliminated by modifying the script to use test-ready compilation (`compile -scan`) starting from the RTL code rather than the Synopsys intermediate format (`.db` file).

Ensure that any post-processing required has been performed on the netlist and/or the SDF timing for compatibility with the cell library. No post-processing was required for the Avant! CB25 reference cell library. The files and scripts supplied expect the gate-level netlist and SDF timing files to be located in the directory `vhd1/utNetlist` (for VHDL) or `verilog/utNetlist` (for Verilog). Create links or copy the files from the `synopsys_ref/netlist` directory to the appropriate `utNetlist` directory before performing gate-level simulations.

4.4.8 BusTalk testbench gate-level simulation

Ensure the testbench is configured for the HCLK period time used in synthesis.

For the VHDL testbench:

To amend HCLK period, amend the Tclk constant in the file BusTalk/vhdl/tbench/timing, for time in ns:
`constant Tclk : time := 16.00 ns; -- 16ns HCLK period for 62.50 MHz clock`

For the Verilog testbench:

To amend the HCLK period, amend the file BusTalk/vhdl/tbench/timing.v, time period in ns:
``define Tclk 16 // 16ns HCLK period for 62.50 MHz clock`

Ensure the environment variables PERIPH and MODELS are set or modify the directory paths.

Ensure links are set appropriately to map the peripheral netlist/HDL and the cell library.

Ensure the memory models and the cell library have been compiled.

Ensure that any post-processing required has been performed on the netlist and/or the SDF timing for compatibility with the cell library.

Edit the libmap file to map uut to the appropriate HDL uutNetlist directory. The netlist must be compiled before compiling the BusTalk testbench as above. To compile the netlist change to the uutNetlist directory:

```
cd $PERIPH/<vhdl|verilog>/uutNetlist
make clean
```

Example simulation log files are supplied and are located in the \$PERIPH/BusTalk/<vhdl|vlog>/tbench/logs. The log files have been generated from simulations of the RTL and netlist for both VHDL and Verilog versions of the peripheral when exercised with the BusTalk functional verification test vectors.

The procedure for running a simulation using the SDRAM controller test vectors is shown below. Ensure that the appropriate test vector file exists in \$PERIPH/BusTalk/bustests/invec (see section 4.4.5).

For VHDL:

Ensure the four link exists to the .bif test vector files in \$PERIPH/BusTalk/bustests/invec (see section 4.4.6):

```
cd $PERIPH/BusTalk/vhdl/tbench
ln -s ../../bustests/invec/Sdram0.bif infile0.bif
ln -s ../../bustests/invec/Sdram1.bif infile1.bif
ln -s ../../bustests/invec/Sdram2.bif infile2.bif
ln -s ../../bustests/invec/Sdram3.bif infile3.bif
```

For Verilog:

Ensure the four links exists to the .sim test vector files in \$PERIPH/BusTalk/bustests/invec (see section 4.4.6):

```
cd $PERIPH/BusTalk/vlog/tbench
ln -s ../../bustests/invec/Sdram0.sim infile0.sim
ln -s ../../bustests/invec/Sdram1.sim infile1.sim
ln -s ../../bustests/invec/Sdram2.sim infile2.sim
ln -s ../../bustests/invec/Sdram3.sim infile3.sim
```

Ensure that the testbench has been compiled before running a simulation.

To perform a back-annotated timing gate-level simulation using ModelSim, invoke the simulator with the appropriate options for minimum or maximum timing, specifying the peripheral instance and the directory path for the post-processed SDF timing file.

The example below runs a maximum SDF timing simulation for the SDRAMC netlist, using the functional verification testbench. Invoke the ModelSim simulator from the tbench directory:

```
vsim -sdfmax uSdram=../../<hdl>/uutNetlist/Sdram_<hdl>.sdf tb_Sdram
```

The cell library path needs specifying as in the modeltech/libmap file using the `-L libvhdl` or `-L libverilog` options as appropriate.

Invoking a run `-all` command in the simulation window will cause the whole of the test pattern to run. The simulation will terminate correctly with the following message to signal completion of the test:

**** Failure: End of test (VHDL) or End of test (Verilog)**

There should be no errors reported during the simulation for a successful run, except during the initialization period. It is advised that simulation transcript files are always checked for any errors or warnings during any subsequent simulations.

To run a gate-level simulation for minimum SDF timing simulation of the SDRAMC, using the functional verification testbench, invoke ModelSim simulator from the `tbench` directory:

```
vsim -sdfmin uSdram=../../<hdl>/uutNetlist/Sdram_<hdl>.sdf -sdfmax tb_Sdram
```

4.4.9 ModelSim error and warning messages

The following errors and warnings may be found in the simulation log files they are not significant.

From VHDL simulation:

```
# ** Warning: /DENRQ1 SETUP High VIOLATION ON enn WITH RESPECT TO cp;
# Expected := 0.472 ns; Observed := 0.303 ns; At : 8 ns
# *Denali* Error: Memory Instance: /tb_sdram/u0_0_sdram Time: 11107 ps
# Setup Violation (tcms=2500 ps) of 65 ps on signal csbar
** Error: HWDATAU : HWDATA is unknown at TIME : 72.000 ns
** Warning: There is an 'U'|'X'|'W'|'Z'|'-' in an arithmetic operand, the result will be 'X'(es)!
** Warning: There is an 'U'|'X'|'W'|'Z'|'-' in an arithmetic operand!
** Failure: Test sequence completed
```

From Verilog simulation:

```
WARNING: [TSCALE] - Module 'Sdram' does not have a `timescale directive in effect, but previous modules do
```

For both VHDL and Verilog simulations, the following warning occurs during the simulation:

```
BWWAREBT: Master may have cancelled the burst before finishing all the transfers....
```

This is benign, the BWWAREBT warning occurs when the test operations deliberately terminate a burst transfer prematurely.

Simulation with Cadence Verilog-XL

4.4.10 Modification of environment variables

Refer to 4.4.1 for information regarding modification of environment variables.

4.4.11 Generating BusTalk test vectors

Refer to 4.4.4 for information on the generation of test vectors.

4.4.12 Linking Cadence to Denali models

The Cadence Verilog simulator must be linked with the Denali models. An example method is given below:

```
cd $PERIPH/BusTalk/vlog/tbench
```

Run the vconfig script provided with the Cadence release. The vconfig script will prompt the user for several options, answer the questions as appropriate for the simulation environment or use the default values.

compile the resulting cr_vlog file and compile and link to the Denali models e.g.

```
gcc -c cr_vlog
```

```
gcc -c $DENALI/verilog/veriusers.c \  
-I$CDS_INST_DIR/tools/verilog/include \  
-o veriusers.o
```

Linking step

```
ld -G veriusers.o $DENALI/verilog/denverlib.o \  
-o libpli.so
```

4.4.13 BusTalk testbench simulation of Peripheral RTL

Example simulation log files are supplied and are located in the \$PERIPH/BusTalk/vlog/tbench/logs directory. The log files have been generated from simulations of the Verilog RTL and netlist versions of the peripheral when exercised with the functional verification test vectors.

Ensure the environment variables PERIPH and MODELS are set or modify the directory paths.

Ensure the Cadence tool has been linked to the Denali models

Ensure that the appropriate test vector file exists in \$PERIPH/BusTalk/bustests/invec.

Ensure the links from the reader directory exist to the .sim test vector files in \$PERIPH/BusTalk/bustests/invec:

```
cd $PERIPH/BusTalk/vlog/reader  
ln -s ../../bustests/invec/Sdram0.sim infile0.sim  
ln -s ../../bustests/invec/Sdram1.sim infile1.sim  
ln -s ../../bustests/invec/Sdram2.sim infile2.sim  
ln -s ../../bustests/invec/Sdram3.sim infile3.sim
```

To perform a simulation using Verilog-XL:

```
vsim -lic_vlog tb_Sdram_SD
or
make vxl_rtl
```

The simulation will terminate correctly with the following message to signal completion of the test:

\$finish at simulation time ...

There should be no errors reported during the simulation for a successful run, except during the initialization period. It is advised that simulation transcript files are always checked for any errors or warnings during any subsequent simulations.

4.4.14 BusTalk testbench gate-level simulation

Ensure the testbench is configured for the HCLK period time used in synthesis.

For the Verilog testbench:

To amend HCLK period, amend the file BusTalk/vhdl/tbench/timing.v, HCLK period time using `define Tclk, for time in ns:

```
`define Tclk          16    // 16ns HCLK period for 62.50 MHz clock
```

Ensure the environment variable PERIPH is set or modify the directory paths.

Ensure links are set appropriately to the cell library.

Simulation log files are supplied and are located in the \$PERIPH/BusTalk/vlog/tbench/logs. The log files have been generated from simulations of the Verilog RTL and netlist versions of the peripheral when exercised with the functional verification test vectors.

Ensure that the appropriate test vector file exists in \$PERIPH/BusTalk/bustests/invec.

Ensure links are made to the .sim files of the Verilog testbench, to the appropriate test vector files in \$PERIPH/BusTalk/bustests/invec, see example below:

```
cd $PERIPH/BusTalk/vlog/reader
ln -s ../../bustests/invec/Sdram0.sim infile0.sim
ln -s ../../bustests/invec/Sdram1.sim infile1.sim
ln -s ../../bustests/invec/Sdram2.sim infile2.sim
ln -s ../../bustests/invec/Sdram3.sim infile3.sim
```

Ensure that any post-processing required has been performed on the netlist and/or the SDF timing for compatibility with the cell library.

To perform a back-annotated timing gate-level simulation using Verilog-XL, invoke the simulator with the appropriate options for minimum or maximum timing, specifying the peripheral instance and the directory path for the post-processed SDF timing file. The example below invokes a maximum SDF timing simulation using functional verification testbench:

```
verilog tb_Sdram.v
+define+NET_MAX
+libext+.v
+incdir+../reader
+incdir+../trickbox
-y ../../../../../../LIBRARIES/avanti/1999.1/cb25/v2.1/verilog/cb25os163/zero
-v ../../../../../../LIBRARIES/avanti/1999.1/cb25/v2.1/verilog/cb25os163/zero/mtb_verilog.v
../../../../verilog/uutNetlist/Sdram_net.v
../common/defs.v
../common/clockgen.v
../common/countdown.v
../bid/cyc_drivers.v
../bid/reset_driver.v
```

```
../bid/split.v
../reader/reader.v
../buswatcher/bus_watch.v
../vr/Reg.v
../vr/linedrv.v
../vr/VIODriver.v
../vr/VRCycDrivers.v
-y ../trickbox
../tbench/ahbslave_tb.v
../tbench/decoder.v
../tbench/defsslave.v
../tbench/timing.v

verilog tb_Sdram.v
+define+NET_MIN
+libext+.v
+incdir+../reader
+incdir+../trickbox
-y ../../../../../../LIBRARIES/avanti/1999.1/cb25/v2.1/verilog/cb25os163/zero
-v ../../../../../../LIBRARIES/avanti/1999.1/cb25/v2.1/verilog/cb25os163/zero/mtb_verilog.v
../../../../verilog/uutNetlist/Sdram_net.v
../common/defs.v
../common/clockgen.v
../common/countdown.v
../bid/cyc_drivers.v
../bid/reset_driver.v
../bid/split.v
../reader/reader.v
../buswatcher/bus_watch.v
../vr/Reg.v
../vr/linedrv.v
../vr/VIODriver.v
../vr/VRCycDrivers.v
-y ../trickbox
../tbench/ahbslave_tb.v
../tbench/decoder.v
../tbench/defsslave.v
../tbench/timing.v

or

verilog tb_Sdram.v +define+NET_MAX -f vx1.net +licq_vx1
verilog tb_Sdram.v +define+NET_MIN -f vx1.net +licq_vx1

or

make vx1_net_max
make vx1_net_min
```

The cell library path has been defined using the \$LIB_VERILOG environment variables.

The simulation will terminate correctly with the following message to signal completion of the test:

\$finish at simulation time

There should be no errors reported during the simulation for a successful run, except during the initialization period. It is advised that simulation transcript files are always checked for any errors or warnings during any subsequent simulations.

4.4.15 Verilog-XL error and warning messages

The following errors and warnings may be found in the simulation log files.

1. Warning! Too few module port connections [Verilog-TFNPC]
This occurs if the synthesis tool has removed unconnected pins from leaf cells, for example, an unused QN flip-flop output.
Synopsys application note SYNTH-480745.html recommends use of `verilogout_show_unconnected_pins = true`, so that an unused pin is written in the instance port map, for example `.QN()`.
2. BWAREBT: Master may have cancelled the burst before finishing all the transfers....
This is benign, the BWAREBT warning occurs when the test operations deliberately terminate a burst transfer prematurely.